

Al-Azhar UNIVERSITY

Faculty of Engineering

Computers and Systems Engineering Department

EXPERIMENT 1 – Introduction to FPGA
(Simple logic, Adders and multipliers using
Verilog language and Quartus software)

OBJECTIVES

- Introduce EDA tools using Quartus and ModelSim software.
- Design simple digital circuits using Verilog.
- Use FPGA board to implement and test digital circuits.
- Design and implement 4 bit Adder circuit.
- Design and implement 4 bit multiplier circuit.

MATERIALS/EQUIPMENT NEEDED

1. Quartus and ModelSim Software.
2. Altera FPGA board

INTRODUCTION

Field Programmable Gate Arrays (FPGAs) are digital ICs (Integrated Circuits) that enable the hardware design engineer to program a customized Digital Logic as per his/her requirements. The term “Field Programmable” implies that the Digital Logic of the IC is not fixed during its manufacturing (or fabrication) but rather it is programmed by the end-user (designer).

FPGA emerged from relatively simpler technologies such as programmable read-only memory (PROM) and programmable logic devices (PLDs) like PAL, PLA, or Complex PLD (CPLD).

It consists of three main parts:

- *Configurable Logic Blocks* — which implement logic functions.
- *Programmable Interconnects* — which implement routing.
- *Programmable I/O Blocks* — which connect with external components.

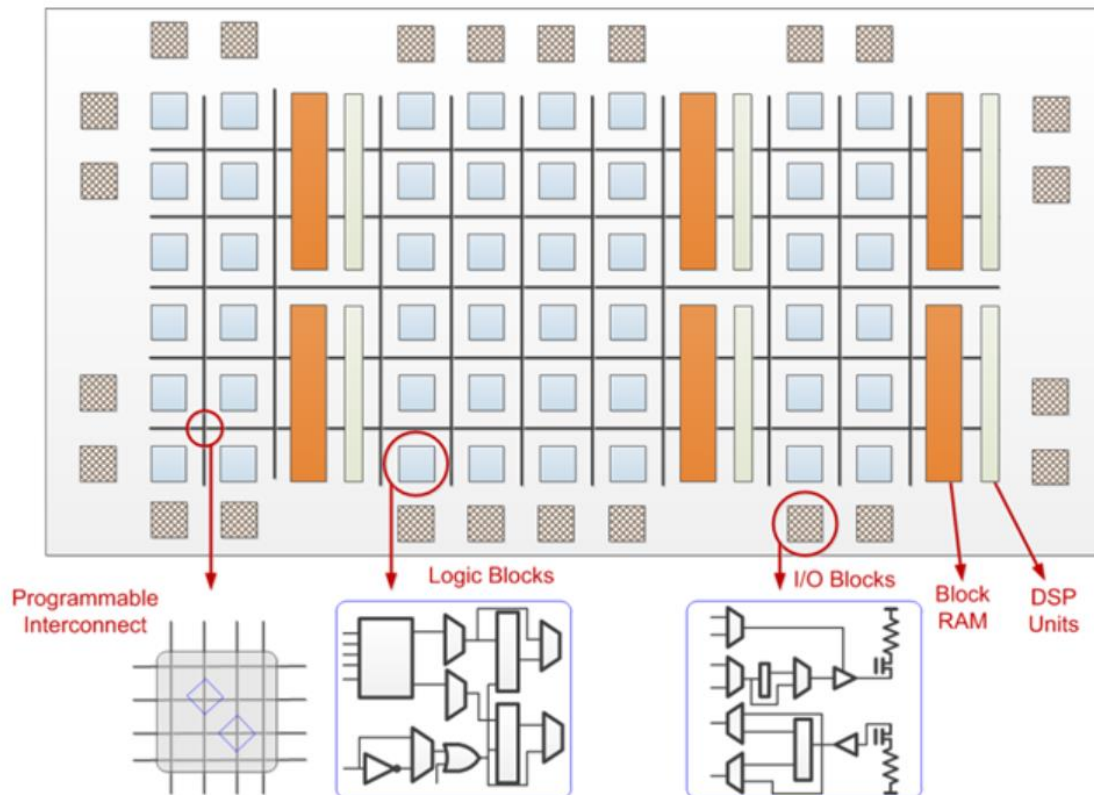
You can think of logic blocks as separate modules like lego blocks which can operate in parallel. Unlike a lego block, a logic block is configurable i.e. its internal state can be controlled and you can hook these together by programming the interconnects in order to build something meaningful.

EXPERIMENT 3 – Introduction to FPGA (Adders and multipliers using Verilog)

This hierarchy of programmable interconnection is used for allocating resources among configurable logic blocks (CLBs); where routing paths contain wire segments of varying lengths that can be connected via anti-fuse or memory-based techniques.

Each CLB is tied to a switch matrix to access the general routing structure. The switch matrix provides programmable multiplexers, which are used to select the signals in a given routing channel and thereby connect vertical and horizontal lines.

Lastly, the I/O blocks (IOBs) are used to interface the CLBs and routing architecture to the external components.



FPGA Architecture

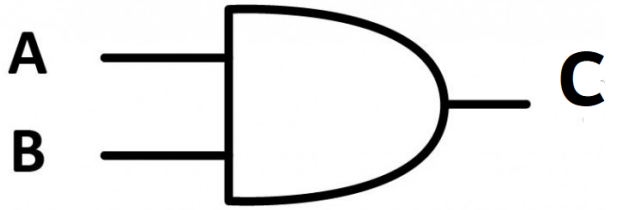
PROCEDURE

Task 1: Simple logic circuits

In task1, an existing Quartus project is opened and Verilog modules for simple logic circuits are written. The Project is compiled, then the generated file is programmed on FPGA board. The functionality of the circuit is tested using Keys and LEDs.

Step 1: Open an existing project.

Open Quartus Software then open an existing project called “my_and.qpf” using File menu ,open project. Open my_and.v file which contains the description of simple “AND” gate.



```
module my_and(a,b,c);  
input a,b;  
output c;  
  
assign c=a&b;  
  
endmodule
```

Step 2: Compile Design

Compile the design using Processing menu, then choose start compilation.

Step 3: Pin Assignments

To know which input and output pins are connected to variables A,B and C, choose pin planner from Assignments menu.

EXPERIMENT 3 – Introduction to FPGA (Adders and multipliers using Verilog)

Pin Planner - E:/sources/Altera_EP4CE/3-Experiment code/Development board routines/Digital logic 01 LED and buttons/my_and/my_and - my_and

File Edit View Processing Tools Window Help

Groups

Named: *

Node Name Direction

<<new...oup>>

Groups Report

Tasks

- Early Pin Planning
- Early Pin Planning
- Run I/O Assignm
- Export Pin Assignm
- Pin Finder...
- Highlight Pins
- I/O Banks
- VREF Groups

Top View
Wire Bond, with Exposed Pad

Cyclone IV E
EP4CE6E22C8

Pin Legend

Symbol	Pin Type
○	User I/O
●	User assi...
●	Filter assi...
●	Unbonde...
●	Reserved ...
●	Other co...
○	DEV_OE
○	DEV_CLR
○	DIFF_n
○	DIFF_p
○	DQ
○	DQS
○	CLK_n
○	CLK_p
○	Other PLL
○	Other du...
○	MSEL0
○	MSEL1
○	MSEL2
○	CONF_D...
○	nCE
○	nCONFIG

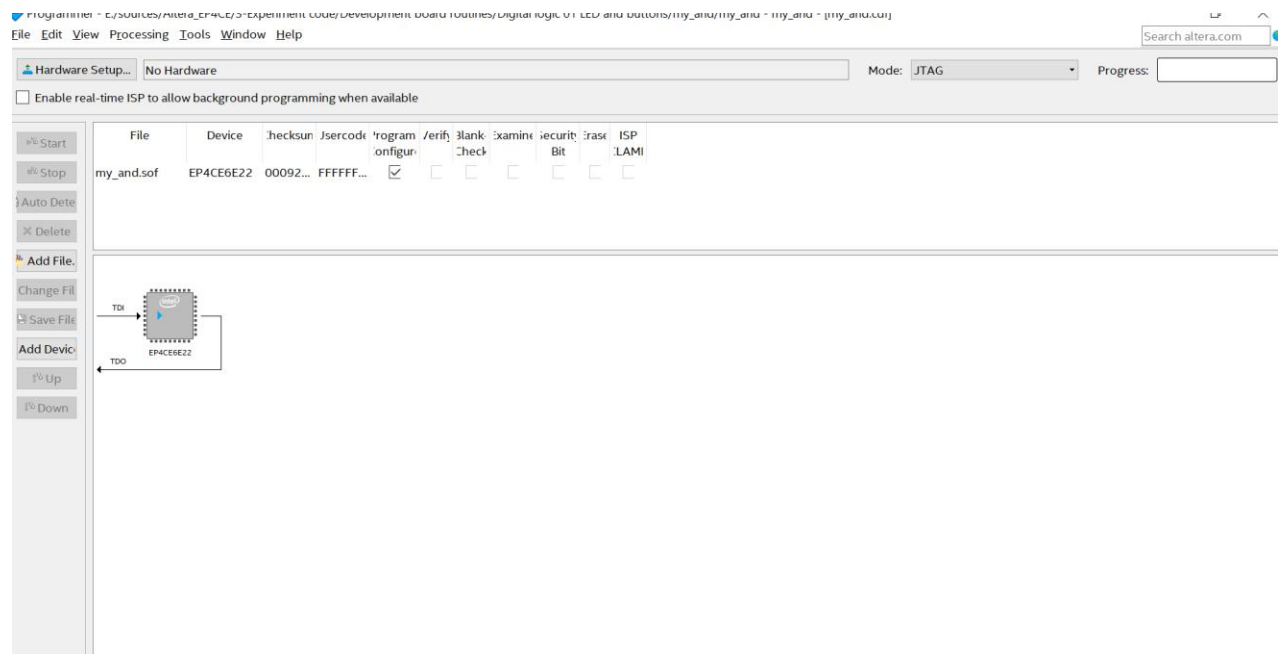
Node Name	Direction	Location	I/O Bank	/REF Grou	ter Locat	'O Standar	Reserved	rent Stre	Slew Rate	fferential P	st Preserv
a	Input	PIN_88	5	B5_N0	PIN_88	3.3...VTTL	8mA...ult				
b	Input	PIN_89	5	B5_N0	PIN_89	3.3...VTTL	8mA...ult				
c	Output	PIN_87	5	B5_N0	PIN_87	3.3...VTTL	8mA...ult	2 (default)			
<<new node>>											

Step 4: Programming

Connected Altera board to USB port then choose programmer from tools menu. Click program button to start programming the generated file to the FPGA board.



EXPERIMENT 3 – Introduction to FPGA (Adders and multipliers using Verilog)



Step 5: Using FPGA board datasheet to determine I/O pins.

To determine which LED or Key is connected to the variables in the design, check the FPGA board datasheet.

The screenshot shows an Excel spreadsheet titled 'Development board pin information.xls'. The spreadsheet lists various components and their corresponding pin numbers. The components are listed in column A, and the pin numbers are listed in column B. The components are: RESET button (pin 25), FPGA_CLK (pin 23), Keys (pin 88), KEY1 (pin 89), KEY2 (pin 90), KEY3 (pin 91), KEY4 (pin 91), Buzzer (pin 110), Dial switch (pin 88), ckey1 (pin 89), ckey2 (pin 90), ckey3 (pin 91), ckey4 (pin 91), LED (pin 87), led1 (pin 86), led2 (pin 85), led3 (pin 84), led4 (pin 84). The pin numbers are listed in column B. The components are listed in column A, and the pin numbers are listed in column B. The components are: RESET button (pin 25), FPGA_CLK (pin 23), Keys (pin 88), KEY1 (pin 89), KEY2 (pin 90), KEY3 (pin 91), KEY4 (pin 91), Buzzer (pin 110), Dial switch (pin 88), ckey1 (pin 89), ckey2 (pin 90), ckey3 (pin 91), ckey4 (pin 91), LED (pin 87), led1 (pin 86), led2 (pin 85), led3 (pin 84), led4 (pin 84).

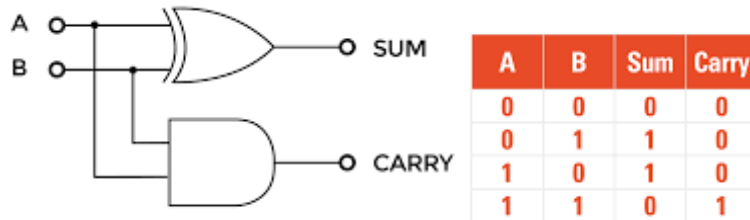
Component	Pin number
RESET button	25
FPGA_CLK	23
Keys	88
KEY1	89
KEY2	90
KEY3	91
KEY4	91
Buzzer	110
Dial switch	88
ckey1	89
ckey2	90
ckey3	91
ckey4	91
LED	87
led1	86
led2	85
led3	84
led4	84

Step 6: Test the functionality of the design.

Using Keys, and switches check the function of the design.

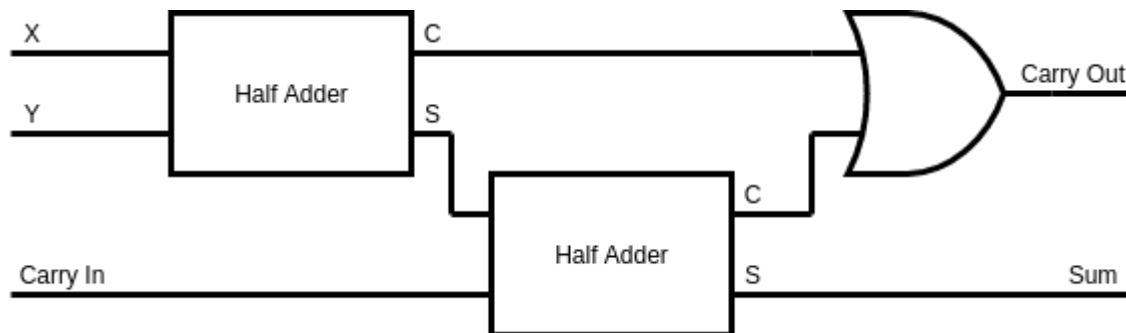
Task 2: Design Half Adder and Full Adder

Using the same steps design , compile and program the half adder and Full Adder circuits



```

module half_adder(A,B,S,CO);
    input A,B;
    output S,CO;
    assign S=A^B;
    assign CO=A&B;
endmodule
    
```

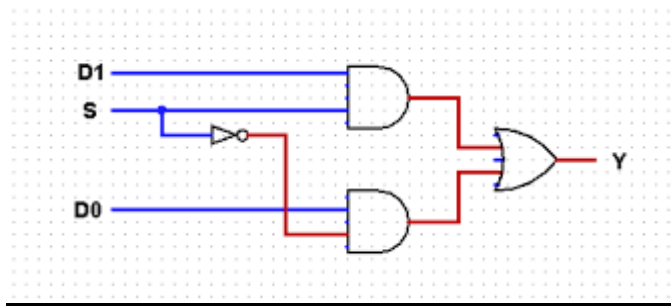


```
module full_adder( A,B, Ci,S,Co );
input      A,B,Ci;
output     S,Co;
wire       C0;
wire       C1;
assign Co = C0 | C1;

half_adder half_adder_0(A,B,A,C0);
half_adder half_adder_0(A0,Ci,S,C1);

endmodule
```

Exercise 1: Design and implement the following logic circuit



Exercise 2: Design and implement 2x1 Mux